Prime Map Plugin — Mimari

X-Plane 11 için katmanlı harita eklentisi · sürüm 0.19 · kodun nasıl çalıştığı, hangi parçanın ne yaptığı ve parçaların birbirine nasıl bağlandığı

Bu belge kodu ilk kez okuyacak bir geliştirici için yazıldı. Kullanım ve test listesi `README.md`'de; yapay zekâ asistanları için çalışma kuralları ve proje durumu `AGENTS.md`'de.

1. Amaç ve temel fikir
2. Büyük resim
3. Bağımlılık kuralı: çekirdek ve X-Plane kabuğu
4. Bir karede ne oluyor?
5. Thread modeli
6. Koordinat sistemleri ve görünüm
7. Katman sistemi
8. Havalimanı verisi (apt.dat)
9. Zemin (DSF sahnesi)
10. Nav verisi, hava yolları, uçak
11. Çizim katmanı (OpenGL 3.3)
12. Arayüz ve fare
13. Harita modları (tema)
14. Ayar ve önbellek dosyaları
15. Dosya dizini
16. Testler ve araçlar
17. Hız ve bellek yöntemleri

1. Amaç ve temel fikir

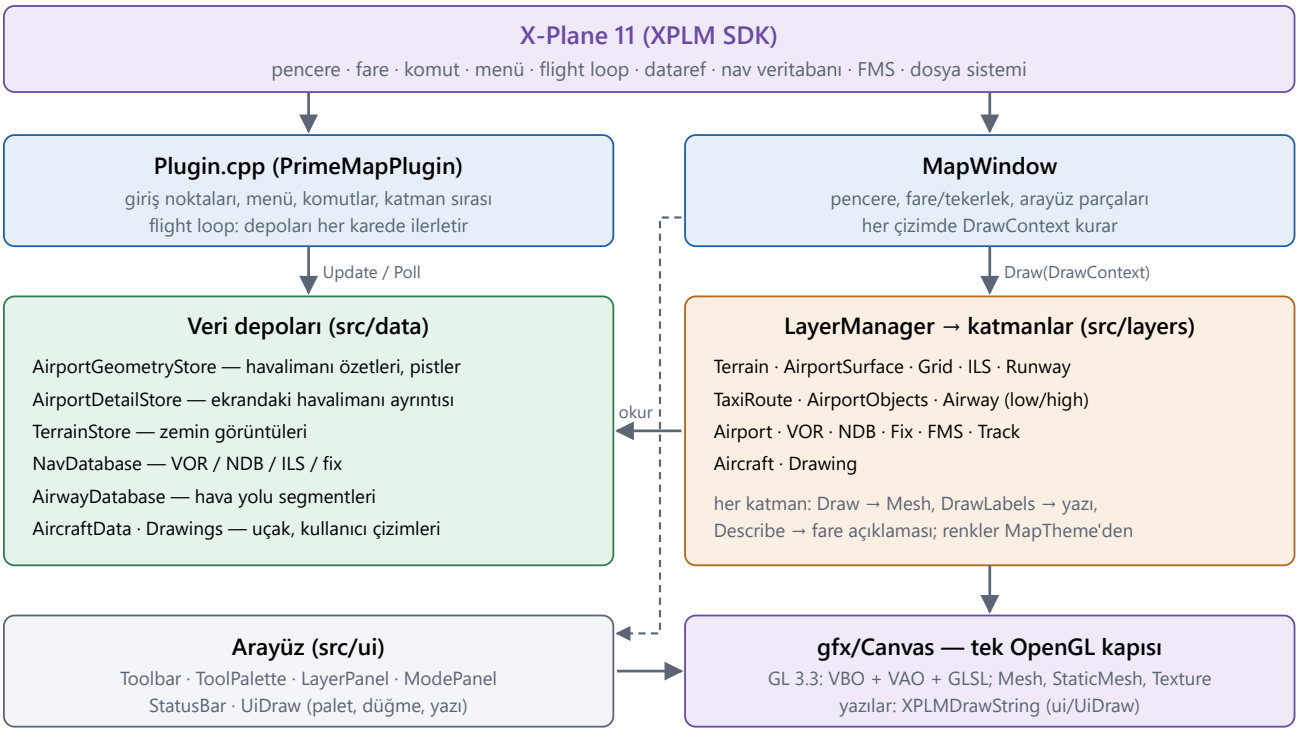
Eklenti, X-Plane'in içinde **kendi penceresinde** çalışan bir harita çizer. Dış harita servisi (OSM, karo sunucusu) kullanmaz; her şey o bilgisayarda kurulu X-Plane verisinden gelir:

- **Zemin:** kurulu DSF sahne dosyaları (arazi türü, su, kabartma).
- **Havalimanları:** apt.dat dosyaları (pistler, apron, taksi yolları, boya çizgileri, kapılar, ışıklar).
- **Navaid'ler:** X-Plane SDK'sının nav veritabanı (VOR, NDB, ILS, fix).
- **Hava yolları:** `earth_awy.dat`.
- **Uçak, FMS rotası:** dataref'ler ve FMS API'si.

Böylece haritada görünen her şey simülatördekiyle birebir aynıdır; yeni sahne kurulunca harita da kendiliğinden güncellenir. Asıl kullanım yeri simülatörlerin **eğitmen istasyonu (IOS)** ekranlarıdır.

Neden X-Plane'in kendi haritasına (`XPLMMap`) katman eklemiyoruz? O pencerenin yan bölmesi kaldırılamıyor, X-Plane'in kendi katmanları açılıp kapatılamıyor ve harita katmanları fare olayı almıyor. Kendi penceremizde bu sınırların hiçbiri yok.

2. Büyük resim



Şekil 1 — Ana parçalar. Düz oklar veri/çağrı yönü; kesikli çizgiler MapWindow'un arayüz parçalarını çizdirmesi.

Okuma sırası önerisi: `Plugin.cpp` → `MapWindow.cpp` (`Draw`) → `Layer.h` → basit bir katman (`layers/GridLayer.cpp`) → `gfx/Mesh.h` ve `gfx/Canvas.h` → ilgilendiğin veri deposu.

3. Bağımlılık kuralı: çekirdek ve X-Plane kabuğu

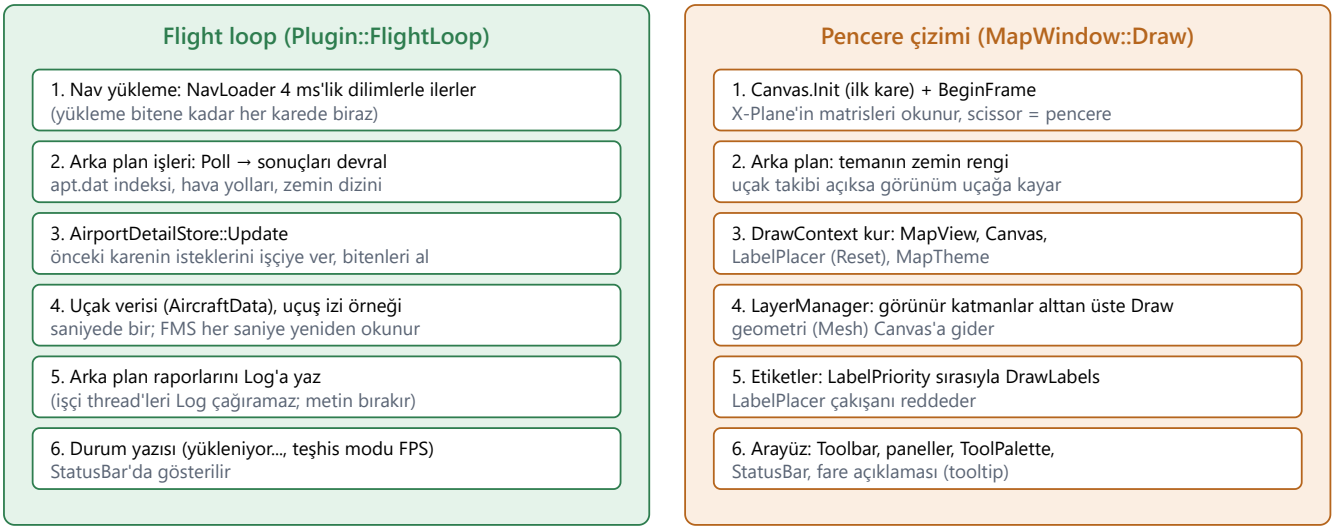
> **Kural:** kodun büyük kısmı X-Plane'e bağımlı **değildir** ("çekirdek"). XPLM başlığı yalnızca ince bir kabukta kullanılır. Bu sayede çekirdek, X-Plane açılmadan testlerde (`tests/`) ve önizleme aracında (`tools/preview`) çalışır.

Kabuk (XPLM kullanır)	Neden
`Plugin.cpp`	Eklenti giriş noktaları, menü, komut, flight loop
`MapWindow.*`	Pencere oluşturma, fare/tekerlek callback'leri, pop-out
`data/NavLoader.*`	SDK'nın nav veritabanını gezer
`data/AircraftData.*`	Dataref'ler (konum, yön)
`layers/FmsLayer.cpp`	FMS API'si
`Log.cpp`	`XPLMDebugString` → Log.txt
`gfx/Canvas.cpp` , `gfx/Texture.cpp`	Grafik durumu, doku numarası
`ui/UiDraw.cpp`	Yazı (`XPLMDrawString`)

CMake bu ayrımı iki listeyle yapar: `CORE_SOURCES` (eklenti + testler + önizleme) ve `PLUGIN_SOURCES` (sadece eklenti; önizleme XPLM fonksiyonlarını `tools/preview/XplmStubs` ile taklit eder). OpenGL çağrısı ise sadece `src/gfx/` içindedir.

4. Bir karede ne oluyor?

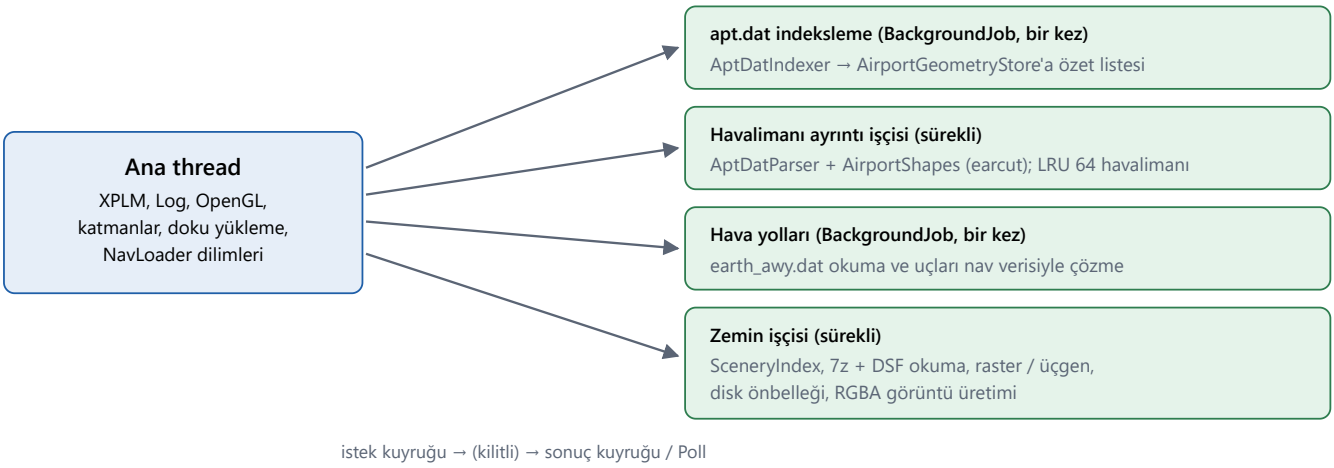
X-Plane eklentiye iki ayrı yerden zaman verir: **flight loop** (her simülasyon karesi) ve pencerenin **draw callback**'i (pencere çizilirken). İkisi de ana thread'de çalışır.



Şekil 2 — Bir karede ana thread'in yaptıkları.

Önemli tasarım noktası: katmanlar veriyi **istekle** alır. Bir katman ekrandaki bir havalimanının ayrıntısına ihtiyaç duyduğunda `AirportDetailStore::Use(i)` çağırır; ayrıntı hazırsa hemen döner, değilse istek kaydedilir ve katman o karede yer tutucu (yanıp sönen kesikli kutu) çizer. Bir sonraki flight loop isteği işçiye verir. Zemin de aynı biçimde çalışır (`TerrainStore::Request` / `PopImage`). Böylece hiçbir yavaş iş çizimi bekletmez.

5. Thread modeli



Şekil 3 — Arka plan thread'leri. Hepsi saf C++; X-Plane'i hiç çağırmaz.

> **Kurallar:** (1) Arka plan thread'inde hiçbir XPLM fonksiyonu, `Log` dahil, çağrılmaz; iş bir rapor metni döndürür, ana thread loglar. (2) Katmanlar sadece ana thread'de çalışır ve depoların ana thread tarafındaki kopyasını okur. (3) İşçiyle paylaşılan kuyruklar `std::mutex` + `condition_variable` ile korunur; tek seferlik işler `BackgroundJob::Poll` ile devralınır (join, yazılanların görünür olmasını garanti eder).

6. Koordinat sistemleri ve görünüm

Sistem	Birim	Nerede kullanılır
Coğrafi	lat/lon (derece)	Tüm veriler, çizimler, ayarlar
Mercator	0..1 (dünya bir kare)	`MapView` merkezi; havalimanı şekilleri referansa göre fark olarak
Boxel	XPLM pencere birimi (global masaüstü)	Bütün çizim ve fare; UI ölçeği %100 değilse pikselden farklı
Piksel	gerçek ekran pikseli	Sadece `glScissor`; dönüşüm `gfx/GlScreen`

`MapView` haritanın "kamerası"dır: Web Mercator, merkez + sürekli zoom (zoom 0'da dünya 256 boxel; her zoom adımında kenar iki katı; en çok 16.5). `ToScreen` / `ToLatLon` dönüşümleri ve imleç etrafında zoom (`ZoomAt`) buradadır. Web Mercator'da ekrana çevirme bir ölçek + kaydırma olduğu için, sabit şekiller GPU'ya bir kez yüklenip her karede sadece bu iki sayı değiştirilerek çizilebilir (`StaticMesh` , bölüm 11).

7. Katman sistemi

7.1 Layer arayüzü

```
class Layer                                  // src/Layer.h
{
    Draw(const DrawContext&)                 // geometri: Mesh uret, Canvas'a ver
    DrawLabels(const DrawContext&)           // yazilar; LabelPlacer'dan yer iste
    Describe(const DrawContext&, x, y)       // fare ustundeyse kisa aciklama (tooltip)
    LabelPriority()                          // kucuk = etiketi once yerlesir
    Id() / Name() / IsVisible() / DefaultVisible()
    AddSubLayer(id, name) / IsSubLayerVisible(i)   // istege bagli alt katmanlar
};

struct DrawContext
{
    const MapView* mView;        // projeksiyon
    Gfx::Canvas*   mCanvas;      // tek cizim kapisi
    LabelPlacer*   mLabels;      // ortak etiket / sembol cakisma onleyici
    const MapTheme* mTheme;      // harita modunun renkleri
    ...                          // pencere siniri, fare konumu
};
```

Etiket yerleşimi: `LabelPlacer` ekrana konan kutuları hatırlar; yeni kutu öncekilerden biriyle çakışıyorsa reddedilir. Katmanlar etiketlerini `LabelPriority` sırasıyla ister (havalimanı 10, pist 15, VOR 20, NDB 30, ILS 40, fix 60, hava yolu 70); semboller de `Reserve` ile kutularını önceden ayırır. Sonuç: hiçbir yazı bir sembolün üstüne binmez, sığmayan yazı yakınlaşınca görünür.

Alt katmanlar: bir katman yapıcısında parçalarını `AddSubLayer("pavement", "Pavement")` ile tanıtır, `Draw` içinde `IsSubLayerVisible(indeks)` ile o parçayı çizip çizmeyeceğine bakar. Panelde katmanın altında açılır liste olarak çıkar; ayar anahtarı `<katman>.<alt katman>`.

7.2 Çizim sırası ve katmanlar

Sıra `Plugin.cpp::CreateLayers()`'tadır (alttan üste). Zoom eşikleri her katmanın .cpp dosyasının başındaki sabitlerdedir.

#	Katman	Veri	Ne çizer / alt katmanlar
1	`TerrainLayer`	TerrainStore	Zemin dokuları (zoom 5+); seviye seçimi, doku LRU (512 MB); alt katmanlar Forests, Built-up areas
1b	`RoadLayer`	TerrainStore (vektörler)	Yollar, demiryolu, enerji hattı (0.17; bölüm 9.8)
2	`AirportSurfaceLayer`	AirportDetailStore	Apron/taksi yolu yüzeyi, kenarları, havalimanı sınırı, boya çizgileri
3	`GridLayer`	—	Enlem/boylam ızgarası, aralık zoom'a göre
4	`NavaidLayer` (ILS)	NavDatabase + pistler	Localizer tüyü pist eşiğinden (pistlerin altında kalsın diye burada)
5	`RunwayLayer`	AirportGeometryStore	Gerçek ölçekli pistler; kaplamalı/kaplamasız/su, blast pad, işaretler, numaralar
6	`TaxiRouteLayer`	AirportDetailStore	ATC taksi ağı (1201/1202), varsayılan kapalı

7	`AirportObjectsLayer`	AirportDetailStore	Işıklar, helipad, beacon, rüzgâr tulumu, kapı/bağlama/hangar, park adları, PAPI/VASI, tabelalar
8–9	`AirwayLayer` ×2	AirwayDatabase	Alçak / yüksek hava yolları, adlar, yön okları
10	`AirportLayer`	NavDatabase + özet	Havalimanı sembolleri ve adları (zoom'a göre seyreltme, kule rengi)
11–13	`NavaidLayer` (VOR, NDB, Fix)	NavDatabase	Semboller, kimlik/frekans; VFR'de VOR pusula gülü
14	`FmsLayer`	XPLM FMS	Uçuş planı (geçilmiş / aktif / kalan bacak)
15	`TrackLayer`	AircraftData	Uçuş izi
16	`AircraftLayer`	AircraftData	Uçak sembolü ve yön çizgisi
17	`DrawingLayer`	Drawings	Kullanıcı çizimleri: noktalar, rotalar, bölgeler, adlar/mesafeler

Ortak yardımcılar: `layers/MapSymbols` (sembol ölçüleri, etiketi sembolün yanına yerleştirme), `layers/AirportVisibility` (ekrana değen havalimanlarını haritanın merkezine yakınlık sırasıyla bulur — ayrıntısı önce merkezdekinin okunsun diye).

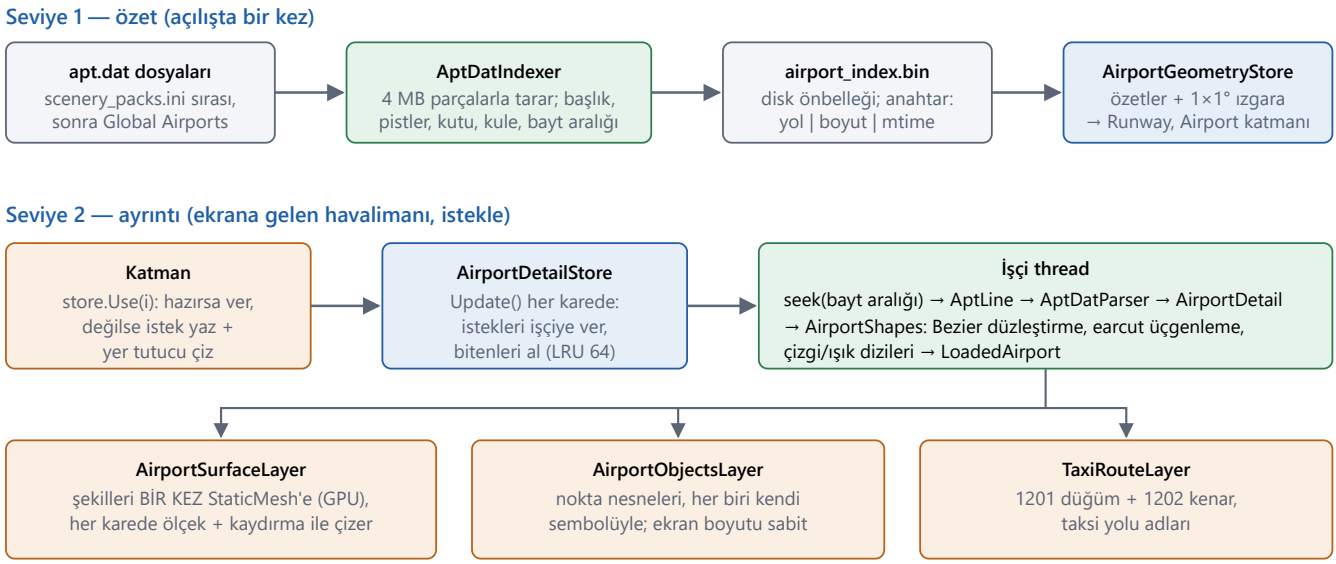
7.3 Yeni katman eklemek

1. `Layer` 'dan türeyen bir sınıf yaz (örnek: `layers/GridLayer.*`); `Draw` , gerekirse `DrawLabels` ve `Describe` .
2. Renkleri sabit yazma, `pContext.mTheme` 'den oku (gerekirse `MapTheme` 'ye alan ekle).
3. `Plugin.cpp::CreateLayers()` 'ta doğru sıraya ekle; dosyayı CMake listesine koy.
4. Mod başına varsayılan görünürlük: `MapTheme.cpp` (`mLayerDefaults`).
5. Panelde grubu: `ui/LayerPanel.cpp` içindeki `kGroups` tablosu.

Görünürlüğün ayar dosyasına kaydı, panelde onay kutusu ve alt katman listesi kendiliğinden gelir.

8. Havalimanı verisi (apt.dat)

apt.dat okuyucu projenin kendisinindir (saf C++, kaynak: XP-APT-1100 spesifikasyonu). Global Airports apt.dat'ı yüzlerce MB olduğu için iki seviyeli çalışır: **özet** her zaman bellekte, **ayrıntı** sadece ekrandaki havalimanları için.

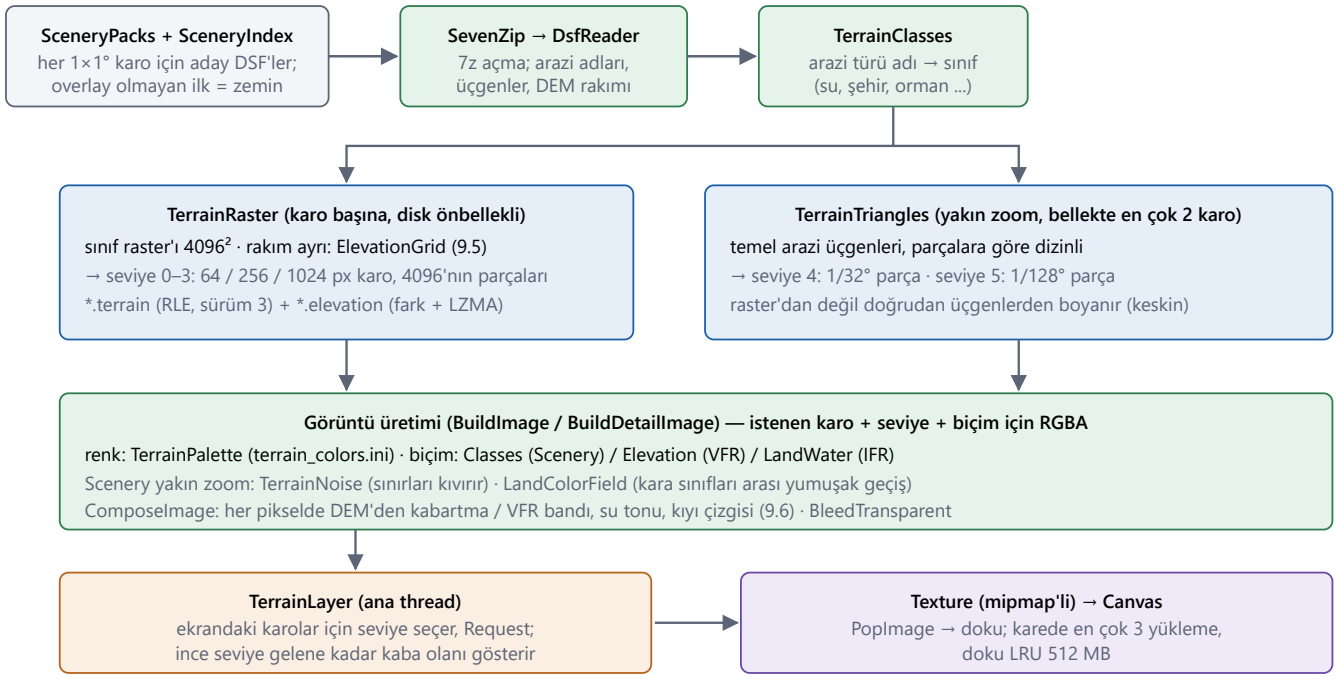


Şekil 4 — apt.dat'tan ekrana: iki seviyeli okuma.

- **Öncelik:** Custom Scenery paketleri `scenery_packs.ini` sırasıyla, sonra varsayılan apt.dat; aynı kimlikte ilk gelen kazanır (X-Plane'in kuralı). `scenery_packs.ini` sadece okunur.
- **AptLine:** sayıları yerel ayardan bağımsız okur (Türkçe Windows'ta `strtod` virgül bekleyebilir).
- **AirportDetail:** apt.dat satır kodlarının birebir karşılığı (100 pist, 102 helipad, 110 yüzey, 120 çizgi, 130 sınır, 111–116 düğümler, 14/15/18/19/20/21, 1201/1202, 1300...).
- **AirportShapes:** hesaplar havalimanı çevresinde metre cinsinden düz bir düzlemde (LocalFrame) yapılır; sonuç noktaları referans noktasına göre Mercator farkı (float) olarak saklanır.
- **Gerçek veri ≠ spesifikasyon:** kodda `GERCEK VERI` notları var (düğüm sonunda 0–3 stil değeri, başlıktaki kule alanı hep 0, Bezier kontrol noktasının yönü...). Biçim değiştirmeden önce bu notları oku.
- **LocalizerMatch:** SDK localizer'ın sadece anten yerini verir; ILS tüyü eşikten başlasın diye uyumlu pist ucu bulunur.

9. Zemin (DSF sahnesi)

Zemin, o bilgisayarda kurulu **her** sahneyle çalışır (bölgeye özel iş yok). Özel sahne global sahneyi ezer, sahne güncellenince harita da güncellenir. Tüm iş eklentinin içinde, arka plandaki zemin işçisinde yapılır.



Şekil 5 — Kurulu DSF sahnesinden zemin dokusuna.

9.1 Seviye seçimi

Seviye	Ne zaman (ekranda derece başına piksel)	Görüntü	Kaynak
0	< 100	64 px, tüm karo	sınıf raster'ı
1	< 400	256 px, tüm karo	sınıf raster'ı
2	< 1600	1024 px, tüm karo	sınıf raster'ı
3	< seviye 4 eşiği	4×4 parçadan biri, 1024 px	sınıf raster'ı
4	yakın	32×32 parçadan biri (1/32°), 1024 px	DSF üçgenleri
5	çok yakın	128×128 parçadan biri (1/128°), 1024 px	DSF üçgenleri

9.2 Önbellek ve senkron kalma

- Her karonun önbellek dosyası, üretildiği **bütün aday DSF'lerin** yol + boyut + değişiklik zamanı listesiyle etiketlidir. Sahne eklenir, çıkarılır, sırası değişir veya güncellenirse etiket tutmaz, karo yeniden üretilir.
- Temel mesh'i olmayan karo (DSF yok ya da sadece overlay): **deniz** mi **veri yok** mu, X-Plane'in dünya tablosu `dsf_files`'tan (bölüm 9.7). Sadece overlay DSF'i olan karo için `.nobase` işareti yazılır (her açılışta yeniden açılmasın).
- Rakım ayrı dosyada (`<karo>.elevation`), aynı anahtarla: raster ile birlikte üretilir, birlikte geçersiz olur. DEM'i olmayan karo için boş rakım dosyası yazılır (kabartmasız karo, yeniden üretilmez).
- Biçim değişirse `TerrainRaster.cpp`'deki `kFileVersion` (şu an 3) ya da `TerrainElevation.cpp`'deki `kFileVersion` (şu an 1) artırılır; eski dosyalar kendiliğinden yeniden üretilir. Palet varsayılanları değişirse `TerrainPalette::kPaletteVersion` artırılır.

- **Disk sınırı (0.18, `data/CacheTrim`):** `[cache] terrain_mb` (varsayılan 1024 MB). Karonun dosyaları (`.terrain` `.elevation` `.vectors` `.nobase`) bir grup sayılır ve birlikte silinir. "Son kullanım" = dosyanın değiştirilme zamanı: önbellekten okunan `.terrain` okununca şimdiye çekilir (Windows'un son erişim zamanı çoğu makinede kapalı). İşçi açılışta bir kez ve yeni karo ürettikten sonra boşta kalınca klasörü tarar; sınır aşıldıysa en eski karoları toplam sınırın %90'ına inene kadar siler. Silme işçi thread'inde yapılır: dosyaları zaten sadece işçi okuyup yazıyor, yarış yok. Silinen karo gerekirse yeniden üretilir.

9.3 Yakın zoom görüntüsü neden ayrı?

Sınıf raster'ı derece başına 4096 pikseldir (enlem 47'de piksel başına ~27 × 18 m); yaklaşınca her piksel ekranda bir blok olur. Bu yüzden seviye 4–5'te görüntü doğrudan DSF üçgenlerinden boyanır. Ama DSF'te arazi türü üçgen başına atandığı için düz renkle boyanınca üçgenler keskin bir mozaik gibi görünür. Bunu iki adım düzeltir:

- **TerrainNoise** ("domain warp"): sınıf okunurken bakılan nokta, enlem/boylama bağlı deterministik bir gürültüyle birkaç on metre kaydırılır; sınırlar kıvrılır. Gürültü coğrafi koordinattan üretildiği için her seviyede ve her parçada aynıdır (dikiş olmaz).
- **LandColorField**: sadece kara renkleri belirli bir yer mesafesinde (`[blend] meters`) normalize bulanıklıkla karıştırılır; su ve kıyı keskin kalır, gölge sonra eklenir. Doku kenar payıyla üretilip kırpılır, komşu parçalar arasında dikiş olmaz.

Ayar anahtarları (`[blend]` , `[warp]` , `[closeup]`) `terrain_colors.ini` 'nin başındaki açıklamada listelidir (dosyayı `Plugin::LoadTerrainColors` yazar).

9.4 Zemin görüntüsünde kullanılan yöntemler

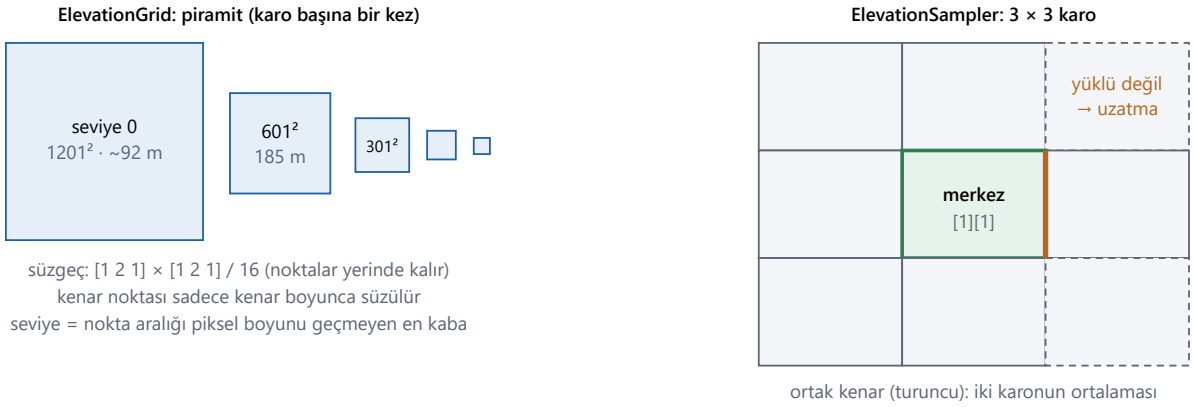
Yöntem	Nerede	Ne işe yarar
Üçgenden doğrudan boyama	`TerrainRaster::BuildDetailImage` , `TerrainTriangles`	Seviye 4 (1/32°) ve 5 (1/128°, ~0,85 m/piksel) parçaları raster'dan büyütülmez, DSF üçgenlerinden yüksek çözünürlükte boyanır: yakında piksel blokları görünmez. Üçgenler parçalara göre dizinlidir, bir parça için sadece ona değen üçgenler gezilir; bellekte en çok 2 karonun üçgenleri tutulur.
Piksel başına kabartma (0.16)	`ElevationSampler` , `ReliefFactor`	Gölge ara bir raster'dan değil, her doku pikselinde DEM'in bikubik (Catmull-Rom) türevinden hesaplanır; piksel boyuna uygun piramit seviyesi kullanılır (bölüm 9.5). Eski 1024^2 gölge ve 512^2 rakım raster'ları kaldırıldı.
Çok yönlü ışık (0.16)	`ReliefFactor` ; `[relief]`	225/270/315/360° ışık, ağırlık 315°'te (ters kabartma yanılsaması olmasın); dik yamaç ışıktan bağımsız biraz koyulaşır (`slope_tone`); kaba seviyede abartma hafif artar (`coarse_boost`).

VFR yükseklik bantları (0.16)	`VfrBandColor` ; `[vfr]`	Sectional sınırları (0 / 1000 / 2000 / 3000 / 5000 / 7000 / 9000 / 12000+ ft). Sınır piksel başına bikubik rakımdan bulunur: her zoomda keskin, kıvrımlı kontur; kenar bir piksel genişliğinde yumuşatılır. İsteğe bağlı ince kontur çizgisi.
Kıyı çizgisi, kıyı şeridi, derinlik tonu (0.16)	`CoastlineStrength` , `DistanceToLand` , `DepthDarkening` ; `[water]`	Suyun tarafında tek piksel kıyı çizgisi (IFR dahil; kaba seviyede kara oranı ondalıklı olduğu için kenarı kendiliğinden yumuşak); kıyıdan açığa ~200 m açık şerit (piksel tavanlı, payla üretilir); açıkta DSF `sea_level` derinliğine göre koyu maviye doğru koyulaşma. Kıyının şekli değişmez.
Saydam piksel doldurma	`BleedTransparent`	Saydam piksellerin rengi en yakın opak komşudan doldurulur (alfa 0 kalır). Doğrusal süzme ve mipmap saydam pikselin siyahını kenara karıştırıp koyu saçak yapmasın diye.
Mipmap'li doku	`Texture::FromRgba(..., true)`	Uzaktan bakınca kumlanma / titreme olmaz (bellek 4/3 kat).
Sınır kıvırma	`TerrainNoise` , `WarpedClassAt` ; `[warp] meters / wavelength` (65 / 250)	Değer gürültüsü + fBm ile bakılan nokta kaydırılır; düz üçgen kenarları doğal sınırlara döner. Genlik pikselin ~0,75'inden küçükse (uzak seviyeler) kendiliğinden etkisizdir. Sadece Scenery (Classes) biçiminde.
Kara renklerinin yumuşak geçişi	`LandColorField` ; `[blend] meters` (150)	Normalize evrişim (kutu bulanıklığı): sadece kara örnekleri karışır, su rengine çekilmez, kıyı keskin kalır; gölge en son eklenir. Doku kenarlarda payla üretilip kırpılır, komşu parçalar arasında dikiş olmaz. Erim 1,5 pikselin altına düşünce plan boş döner (uzak seviyelerde eski görünüm değişmez).
Erim tavanı	`[blend] max_pixels` (5)	Yakın seviyelerde yumuşatma erimi doku pikseliyle sınırlanır; havalimanı ayrıntılarının göründüğü zoomlarda zemin bulanıklaşmaz.
Yakın zoom sınıf karışımı	`CloseupMixAt` , `MixedColors` ; `[closeup] class_mix` (0,3)	Piksel boyu 50 m'den küçüldükçe (5 m'de tam oran, arası logaritmik) kara sınıf renkleri ortak bir kara tonuna yaklaşır; yakında renk mozaiği sakinleşir. Su, kar ve "None" karışmaz.

Palet sürümü	`TerrainPalette::kPaletteVersion` (2)	Varsayılan renkler değişince dosyadaki eski renkler atılır, dosya yeni varsayılanlarla yeniden yazılır (shading / blend korunur).

9.5 Rakım: tam çözünürlük, piramit, komşu karolar (0.16)

0.15'e kadar rakım 512^2 (~200 m), gölge 1024^2'lik ara raster'dan okunuyordu: yakında bulanık, uzakta seyrek örnekleme yüzünden parazitli (aliasing). 0.16'da DSF'in DEM'i **tam çözünürlükte** (1201 × 1201 nokta, ~92 m) saklanır ve her doku pikseli onu kendi boyuna uygun bir **piramit seviyesinden** okur.



Bikubik okuma 4×4 nokta ister; kenarda bir kısmı komşu karodadır. Komşu yüklüyse onun noktaları, değilse merkezden aynalı doğrusal uzatma (kenarda tek taraflı fark). Komşu sonradan gelirse doku yenilenir.

Şekil 5b — Rakım piramidi ve komşu karolu örnekleme.

- **Saklama:** `ElevationCodec` — satır içi fark (*delta*), bayt düzlemlerine ayırma, LZMA (seviye 4, tek thread; `third_party/lzma` 'ya aynı SDK'nın sıkıştırıcısı eklendi). Gerçek DEM'de 2,9 MB → ~0,75 MB; sıkıştırma ~0,25–0,3 s (karo üretilirken bir kez), açma ~50 ms. Seviye 5 normal moda geçer: %10 daha küçük, 5 kat yavaş (ölçüldü).
- **Komşu kuralı:** komşu karonun rakımı sadece bellekten ya da önbellekten alınır, komşu için DSF açılmaz. Komşusu henüz üretilmemiş bir doku `TerrainImage::mMissingNeighbors` listesiyle işaretlenir; komşu hazır olunca (`TerrainStore::HasElevation`) katman o dokuyu yeniden ister, yenisi gelince eskisinin yerine koyar.
- **GERÇEK VERİ:** komşu karoların ortak kenar noktaları **aynı değil** (demo DSF'lerinde ortalama 1–2 m, en çok ~30 m fark). Bu yüzden ortak kenar ve köşede var olan bütün karoların ortalaması alınır; iki taraf sınırda aynı yüksekliği görür, gölge ve VFR bant sınırında basamak olmaz.
- **Beklenti:** DEM ~92 m aralıklıdır. z15+ zoomda (piksel ~3 m) kabartma yumuşak görünür; bikübik okuma bloğu yok eder ama yeni ayrıntı yaratamaz. Simülatörün kendi zemini de aynı veriden oluşur: bu, X-Plane verisinin sınırı.

9.6 Görüntünün son adımı: ComposeImage (0.16)

İki üretici (kaba seviyeler: `BuildImage` , yakın zoom: `BuildDetailImage`) önce aynı ara sonucu doldurur: payla genişletilmiş alanda her pikselin **kara oranı** (0..1, veri yoksa −1), **kaplama** (alfa) ve çıktı piksellerinde Scenery kara rengi. Sonra tek bir fonksiyon her seviyede aynı formülle bitirir; bu yüzden ton seviyeden seviyeye tutarlıdır.

```
payli alan (kara orani)  -> CoastlineStrength (4 komsu)   -> kiyi cizgisi (suyun tarafinda 1 piksel)
                         -> DistanceToLand (3-4 chamfer)  -> kiyiya uzaklik (serit)
her cikti pikseli:
   rakim + egim (ElevationSampler, uygun seviye)
   kara : Classes = sinif rengi | Elevation = VFR bandi (VfrBandColor) | LandWater = duz renk
          x ReliefFactor (Scenery ve VFR)
   su   : su rengi x (1 - DepthDarkening(sea_level)) + kiyi seridi aciklik   (IFR'da duz su)
   renk = kara * oran + su * (1 - oran);  kiyi cizgisi ustune;  alfa = kaplama
```

Ayar (`terrain_colors.ini`)	Varsayılan	Ne

`[relief] multi_directional`	1	4 yönlü ışık (0: tek ışık 315°)
`[relief] exaggeration / coarse_boost / slope_tone`	2,5 / 0,35 / 0,12	Eğim abartması; kaba seviyede artış üssü; dik yamaç koyuluğu
`[relief] flat_slope`	0,02	Bu eğimin altı yumuşakça bastırılır: $s \cdot s^2/(s^2+s_0^2)$. DEM tam metre saklandığı için düz ovada 1 m'lik basamaklar (~%1 eğim) dalga gibi görünüyordu; %2'de basamak etkisi %23'e iner, %5 yamaç %86'sını, %15+ tamamını korur (ölçüldü)
`[water] coastline / coastline_strength`	1 / 0,5	Kıyı çizgisi
`[water] band_meters / band_max_pixels / band_strength`	200 / 24 / 0,10	Kıyıdan açığa açık şerit, kıyı çizgisinin bir piksel arkasından başlar (0 = kapalı)
`[water] depth_tone / depth_full_meters`	0,25 / 200	Derinliğe göre koyulaşma; kanal ağırlıkları R 1,0 · G 0,8 · B 0,5 (eşit koyulaşma suyu griye çeviriyordu)
`[vfr] bands / contour_lines / contour_strength`	1 / 0 / 0,35	Bantlar (0: eski rampa); ince kontur
`[vfr] band_1 .. band_8`	sectional	`<alt sınır feet> R G B`; sınırlar artan olmalı

GERÇEK VERİ (`sea_level`, 256^2 hücre merkezli, satır 0 = güney): denizde deniz tabanı yüksekliği gibi davranır (Puget Sound en derin −242 m). Verisi olmayan deniz hücresi −10 (Juneau kanallarının çoğu), göllerde ≥ 0, karadaki değerlerin anlamı yok. DSF'teki `bathymetry` katmanının adı var ama verisi yok. Biçim belgelenmemiş; anlam ölçümden çıkarıldı, bu yüzden derinlik tonu sadece −10'dan derin hücrelerde uygulanır ve kapatılabilir.

9.7 Deniz mi, veri yok mu: dsf_files (0.16)

- `Resources/dsf_files` (`WorldTileMap`): 5 bayt başlık (anlamı bilinmiyor) + 360 × 180 bayt, boylam öncelikli, `indeks = (boylam + 180) × 180 + (enlem + 90)`; 255 = X-Plane'in tam global sahnesinde bu karonun DSF'i var, 0 = yok (açık deniz). Kurulu 35 demo karosu ve bilinen kara/deniz noktalarıyla doğrulandı.
- Temel mesh'i olmayan karo: tabloda 0 ise **deniz**, 255 ise **veri yok** (kara ama sahnesi bu bilgisayarda kurulu değil). "Veri yok" karosu deniz gibi boyanmaz; temanın nötr `mNoData` rengiyle doldurulur.
- Güvenlik: tablo açılışta Global Scenery'de kurulu karolarla doğrulanır (hepsi 255 olmalı, %2 pay). Dosya yoksa, boyutu farklıysa ya da doğrulama tutmazsa eski kural çalışır (karonun 10°'lik klasörü kuruluysa deniz). Sonuç Log.txt'ye yazılır: `world tile map: ... (N mismatches)`.
- `Resources/map data/water/*.shp` (X-Plane'in kendi haritasının suyu) incelendi: DSF kıyısının sadeleştirilmiş hâli, sadece büyük su kütleleri. Simülatördeki su DSF olduğu için esas DSF; shapefile ileride sadece kurulu olmayan karolar ve hızlı ilk görüntü için düşünülebilir (Aşama 4).

9.8 DSF vektörleri ve X-Plane paleti (0.17)

- **Okuma:** `DsfReader::ForEachVector` poligonları 16 bitlik (`POOL`), ağ zincirlerini 32 bitlik (`PO32` / `SC32`) havuzlardan okur. `TerrainVectors` tanım adına göre sınıflar: `.for` → orman / çalı / sulak / tarla, autogen `urban_` / `ind_` / `com_` → yerleşim, `park_` → çimen; yollar alt türe göre otoyol / ana / tali / yerel / demiryolu / enerji hattı. Noktalar Douglas-Peucker ile 2,5–4 m'ye sadeleştirilir.
- **Dışlama:** aynı karonun overlay DSF'lerinden (Global Airports) `sim/exclude_for|obj|fac|net` (`batı/güney/doğu/kuzey`). Yollar dışlama alanında kesilir; alanlar boyanırken piksel piksel atlanır.
- **Alanlar zemine işlenir:** karo üretilirken 4096^2'lik ayrı bir alan katmanına taranır (çift-tek kuralı, sadece karada); yakın zoom parçalarında doğrudan poligonlardan. Görüntü anahtarındaki maske (`TerrainImageKey::mAreas`,

Terrain alt katmanları Forests / Built-up areas) uygulanıp uygulanmayacağını seçer. Scenery'de sınıf rengi olarak, VFR/IFR'da yerleşim temanın `mBuiltUp` rengine karışarak.

- **Yollar:** `RoadLayer` vektörleri `TerrainStore::RequestVectors / PopVectors` ile alır, karo başına tür başına bir `StaticMesh` (çizgi) yükler; renk temadan (`mRoads`), kalınlık ekran pikseli. Önbellek `*.vectors` (16 bit nokta + LZMA).
- **Palet (Aşama 3):** `TerrainPaletteSource` `terrain10/*.ter` dosyalarının `BASE_TEX` DDS dokularının küçük bir mip seviyesini çözer (DXT1/3/5, BGRA), sınıf başına ortalar (~3.450 doku). `[palette] source = xplane` varsayılan; sonuç önbelleğe yazılır.

9.9 Aşama 4: öncelik, önceden yükleme, bölge önbelleği (0.19)

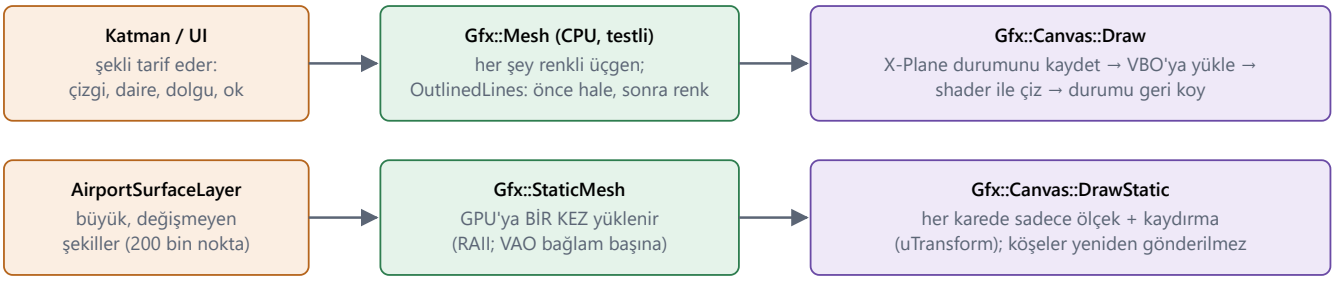
- **Merkezden kenara:** `TerrainLayer` her çizimde `TerrainStore::SetFocus` 'a harita merkezini verir. `PickRequest` sırası: en son istenen kare → kaba seviye → merkeze yakın parça.
- **Arka plan kuyruğu:** ekranın isteği yokken işçi `ServeBackgroundTile` ile bir karonun disk önbelleğini (raster, rakım, vektör) hazırlar; geçici nesnelere çalışır, bellek LRU'sunu ve `IsIdle` 'ı etkilemez (harita resmi boşuna yenilenmez). Önce bölge listesi, sonra önceden yükleme listesi.
- **Önceden yükleme:** eklenti 5 s'de bir uçağın gidiş yönünde 120 NM'ye kadar ve FMS rotasının kalan kısmında (`FmsLayer::RouteAhead` , 10 NM örnekleme) en çok `[cache] prefetch_tiles` (20) karoyu verir.
- **Görünen alanı önbellekle:** komut / menü. Kurulu karolar × ~3,5 MB önbellek sınırının %80'ini aşarsa başlamaz (yoksa `CacheTrim` yeni karoları silerdi). İlerleme ve iptal alt şeritte / menüde.
- **İkinci işçi yok (karar):** raster / rakım / vektör LRU'ları kilitsiz ve tek işçinin; bölmek bunları paylaşılan ve kilitli yapmayı gerektirirdi, ölçülmüş bir ihtiyaç yok. İşçi düşük thread önceliğinde.

10. Nav verisi, hava yolları, uçak

- **NavLoader** X-Plane'in o an yüklü nav veritabanını SDK ile gezer (`XPLMGetFirstNavAid` / `XPLMGetNextNavAid` / `XPLMGetNavAidInfo`). XPLM sadece ana thread'den çağrılabildiği için iş flight loop'ta 4 ms'lik dilimlere bölünür. Dosya okumaz: haritadaki navaid'ler simülatördekiyle aynıdır.
- **NavDatabase** kayıtları tutar ve her tür için ayrı bir **1×1° ızgara** (`GeoGrid`) kurar; katmanlar sadece ekrandaki hücrelere bakar (yüz binlerce fix var).
- **AirwayDatabase** `earth_awy.dat` 'ı (XP11, 1100 biçimi) okur; önce `Custom Data` , sonra varsayılan. Satırda uçların koordinatı yok: kimlik + bölge + türle nav verisinden çözülür.
- **AircraftData** kullanıcı uçağının konumu ve yönünü dataref'lerden okur; uçak katmanı, iz ve "uçağı takip et" aynı veriyi kullanır.
- **Drawings** kullanıcı çizimlerinin modeli (lat/lon olarak saklanır, `map_drawings.txt`). `Geo` büyük daire mesafesi, ara nokta ve alan hesabını yapar.

SDK gerçekleri: ILS yönü **gerçek** yöndür (manyetik değil); TACAN frekansı kanal numarasıdır; "----" kimlikli yer tutucu ILS kayıtları filtrelenir. VOR pusula gülü `NavRecord::mVariation` ile döner.

11. Çizim katmanı (OpenGL 3.3)



Şekil 6 — İki çizim yolu: her karede üretilen Mesh ve GPU'da kalıcı StaticMesh.

Dosya	Görev
`gfx/Mesh.*`	CPU'daki renkli üçgen listesi. Modern GL'de `glLineWidth` güvenilmez olduğu için kalın çizgi, daire, çerçeve hepsi burada üçgene çevrilir; kalınlıklar boxel. `OutlinedLines` okunurluk için önce koyu/açık hale, sonra renkli çizgi.
`gfx/Canvas.*`	Tek çizim kapısı: VBO + VAO + GLSL 330. Her çizimde X-Plane'in program/VAO/VBO/cull durumunu kaydedip geri koyar; matris = GL_PROJECTION × GL_MODELVIEW (BeginFrame'de okunur). Dokulu şeritler (zemin) de buradan.
`gfx/StaticMesh.*`	GPU'da kalıcı şekil; `DrawStatic` ile ölçek + kaydırma.
`gfx/ContextVertexArray.*`	Pop-out pencere ayrı OpenGL bağlamında çizilir. Buffer, doku ve shader'lar bağlamlar arasında paylaşılır ama **VAO paylaşılmaz**; başka bağlamın VAO'sunu bağlamak X-Plane'i çökertir. Bu sınıf VAO'yu o anki bağlam için (gerekirse yeniden) oluşturur.
`gfx/GlApi.*`	Windows'un `opengl32.lib` 'i sadece GL 1.1 verir; 3.3 fonksiyonları ilk çizimde `wglGetProcAddress` ile yüklenir. Bağlam değişimini (`ContextGeneration`) izler.
`gfx/Texture.*`	RAII doku; numara `XPLMGenerateTextureNumbers` , bağlama `XPLMBindTexture2d` . İsteğe bağlı mipmap.
`gfx/GlScreen.*`	Boxel → piksel dönüşümü (GL matrisinden) ve pencere alanıyla kırpma (scissor).

> **X-Plane ile birlikte yaşamak:** `XPLMDrawString` doku açar, X-Plane GL_CULL_FACE'i açık bırakabilir; bu yüzden her geometri çiziminden önce `XPLMSetGraphicsState` çağrılır ve cull kapatılır (Canvas bunu kendisi yapar — Canvas dışından çizim yapma). Yazılar sadece ASCII çizilebilir (Türkçe harf, derece işareti yok).

11.1 Harita resmi önbelleği (0.19, `MapImageCache`)

Harita her karede baştan çizilmez. Katmanlar bir FBO dokusuna (`gfx/RenderTarget`) çizilir; her karede bu doku tek bir dörtgen olarak (saydamlıksız) ekrana basılır.

```
OnDraw:  PrepareAll (hazir dokular / vektorler)  -> yeni veri?
         SceneSignature (mod, katmanlar, cizim)    -> degisti mi?
         MapImageCache::Draw
             gerekirse: BeginOffscreen -> arka plan + LayerPass::Cached -> EndOffscreen
             eski resmi kaydir (tam piksel) / olcekle -> bas
         LayerPass::Overlay (ucak, iz, yarim cizim) -> tooltip -> kontroller -> FlushText
```

- **Pay:** doku pencereden her yanda %25 büyük; kaydırırken ve takipte kenar boş kalmaz. Gösterim: resmin noktası p → `M + (c0 − c)·W + (p − M0)·W/W0` ; sadece kaydırmada tam piksele yuvarlanır (bulanıklık yok).

- **Ne zaman çizilir:** hemen (ilk kez, boyut / UI ölçeği, imza değişti, resim pencereyi kaplamıyor); hız sınırında `[render] max_fps` (görünüm değişti, yeni veri, veri yükleniyor); güvenlik için en geç 1 s.
- **Görünüm:** resmin `MapView`'ı aynı zoom, geniş viewport; merkezi, içerik pencereyle birebir aynı yere düşecek şekilde kaydırılır. Etiketler pencerenin boş alanına yerleşir (payda etiket yok).
- **X-Plane ile yaşamak:** `BeginOffscreen` X-Plane'in draw/read framebuffer'ını, viewport'u, scissor'ı ve temizleme rengini kaydedip geri koyar. FBO nesnesi bağlamlar arası paylaşılmaz: pop-out'ta `RenderTarget` o bağlam için yenisini aynı dokuya bağlar.
- **Yedek:** FBO kurulamazsa ya da `[render] cache = 0` / `[text] renderer = xplane` ise harita eskisi gibi her karede doğrudan çizilir.
- **Katman kuralları:** her karede gereken iş `Layer::PrepareFrame`'de; her karede değişen çizim `Layer::IsOverlay` ile üstte; pencere kenarına yapışık parçalar (ızgara yazıları) `Layer::DrawOverlay`'de. Havalimanı yükleniyor çerçevesinin yanıp sönmesi bu yüzden kaldırıldı.

11.2 Yazı çizici (0.19)

- `gfx/FontAtlas`: X-Plane'in `Resources/fonts/DejaVuSans.ttf`'i (dağıtmıyoruz) stb_truetype ile (`third_party/stb`, kamu malı) bir RGBA atlasa basılır: ASCII, Latin-1, Türkçe harfler. Boyut büyük harf yüksekliğiyle verilir = X-Plane yazısının yüksekliği (`XPLMGetFontDimensions`), yerleşim değişmez.
- `ui/AtlasTextRenderer` (`TextRenderer` arayüzü; `Ui::SetTextRenderer`): harfleri tam piksele oturtur, UI ölçeği değişince atlası yeniden basar, köşeleri `Canvas::QueueText`'e verir. Canvas bir sonraki şekilden önce ya da `FlushText`'te hepsini tek çağrıyla çizer.
- Yedek: `XPLMDrawString` (`[text] renderer = xplane` ya da font okunamazsa).

12. Arayüz ve fare

Pencerenin tamamı haritadır; kontroller haritanın üstünde ince ve yarı saydamdır. Arayüz parçaları **sadece çizer ve tıklamanın neye denk geldiğini söyler** (`HitTest`); ne yapılacağına `MapWindow` karar verir. Ortak palet ve çizim yardımcıları `ui/UiDraw`'dadır (`Ui::k...` renkleri).

Parça	Ne
`Toolbar`	Üst şerit: Layers · Map: <mod> | Follow / Center on aircraft · Go to airport | Draw tools … Pop out · Hide. Gruplar arasında ayırıcı; dar pencerede pencere düğmeleri ikinci satıra iner.
`ToolPalette`	Yüzen çizim paleti: Pan / Point / Route / Region + Finish / Undo / Clear. Başlıktan sürüklenir, seçili araca katlanır; yeri sağ üst köşeye göre `[tools]`'a kaydedilir.
`LayerPanel`	Grup › katman › alt katman listesi; üç durumlu kutu (hepsi / hiçbiri / karışık), ok ile aç/kapa, tekerlekle kaydırma. Gruplar `kGroups` tablosunda.
`ModePanel`	Harita modu seçimi (tek seçim).
`AirportSearchPanel`	(0.18) "Go to airport": yazı kutusu + en iyi 8 sonuç (`data/AirportSearch`: kimlik / ICAO tam → önek → ad kelime başı → ad içinde; eşitlikte uzun pistli önce). Seçilince `MapWindow::GoToAirport` haritayı ortalar ve havalimanını sığdıran zoom'u seçer (`ZoomToFitAirport`); **uçağa dokunmaz** (kullanıcı kararı), takibi kapatır. Panel açıkken pencere klavye odağını alır (`XPLMTakeKeyboardFocus`); haritaya tıklama, Esc, panelin kapanması veya pencerenin gizlenmesi odağı X-Plane'e geri verir.
`StatusBar`	İmleç koordinatı, zoom, durum mesajı, ölçek çubuğu (NM). 0.18: `TerrainLayer::PendingTextureCount` > 0 (son karede istenen seviyede olmayan doku) ve zemin işçisi boşta değilken `Loading terrain: N images` yazar; işçi boştaysa yazmaz (üretilemeyen parça yazıyı takılı bırakmasın).

Fare akışı (MapWindow)

```
basildi  -> once kontroller: HandleControlClick (toolbar, paneller, palet basligi)
            degilse haritaya basildi (mPressOnMap)
hareket  -> 4 px'ten fazla: surukleme = haritayi kaydir (ucak takibi kapanir)
            palet basligindan tutulduysa: paleti tasi
birakti  -> hic hareket yoksa tiklama: HandleMapClick
            Point / Route / Region modunda nokta / kose ekler
sag tik  -> yarim rota / bolgeyi bitirir
tekerlek -> imlecin altindaki nokta sabit kalarak zoom (takipteyken ucagin etrafinda);
            imlec katman panelindeyse paneli kaydirir
```

Tıklama asla alttaki pencereye geçirilmez (SDK dokümanına göre fare takibini bozar). Pop-out modunda imleç konumu cursor callback'ten okunur.

13. Harita modları (tema)

`src/MapTheme.*` beş modu tanımlar. Her mod bir `MapTheme`: arka plan ve su rengi, zemin biçimi, hale rengi, açık/koyu tema bayrağı, ızgara, havalimanı/navaid/hava yolu/pist renkleri ve mod başına varsayılan katman görünürlüğü. `DrawContext::mTheme` her zaman doludur; katmanlar renkleri buradan okur.

Mod	Zemin biçimi	Özellik
Scenery	Classes — arazi sınıfları + kabartma	Koyu tema
VFR	Elevation — rakım rampası + kabartma	Kuleli havalimanı mavi, kulesiz macenta; VOR pusula gülü
IFR Low	LandWater — iki düz renk	Açık tema; alçak hava yolları, VOR/NDB/ILS/fix
IFR High	LandWater	Sadece büyük havalimanları, yüksek hava yolları
Dark (0.18)	LandWater — siyah kara, koyu lacivert su	Uçuş kontrol ekranı tarzı minimal: havalimanı, pist, navaid, ILS, hava yolları; ızgara / yol / nesne yok

Varsayılan katmanlar (0.18): `MapTheme::mLayerDefaults` bir `std::span`; `MapTheme.cpp` başındaki tablolar her katman ve alt katman (`katman.alt`) için mod başına değer verir. Scenery / VFR / IFR tabloları kullanıcının X-Plane'de kaydettiği ayarlardan üretildi. Tabloda olmayan katman `Layer`'ın kendi varsayılanını alır; `map_settings.ini`'deki değer her zaman önceliklidir.

Mod değişince (`MapWindow::SetMapMode`) eski modun katman görünürlüğü kaydedilir, yenisininki yüklenir. Zemin görüntü anahtarı biçimi de içerir (`TerrainImageKey::mStyle`), böylece her modun dokusu ayrı üretilip önbelleklenir.

13b. Prime Map API (0.20; 0.22'de ekip kalıbına geçti)

Harita bir kütüphane (`prime_map`) oldu; tek genel başlık `PrimeMap.h` (0.22'de repo köküne taşındı; sadece standart C++). Ayrıntılı referans: `docs/PrimeMap_API.pdf`.

```
eski (0.19)                          yeni (0.20)
Plugin.cpp  (veri + menu + komut)    engine/MapData.*      veri depolari, yukleme, prefetch, precache,
palet, flight loop
                                     plugin/PluginMain.cpp menu, prime/map/* komutlari, ini dosyalari
(sadece API ile)
MapWindow   (pencere + ayar dosya)   MapWindow             motor: pencere (Embedded/Standalone), cizim,
fare, UI;
                                                           ayarlar bellekte + revizyon sayaci
MapWindow::ApplyLayerPanelHit        engine/LayerTable.*   MapLayer enum <-> anahtar tablosu + panel
kurallari
                                     api/PrimeMap.*        genel API kopru (coklu harita; durum
MapLibrary nesnesinde)
```

- **Fare yönlendirmesi:** basmada önce UI'a sorulur; almazsa ve fare kapalıysa basma-sürükleme-bırakma hepsi 0 döner (alttaki pencereye). Tekerlek ve sağ tık da aynı.
- **Dosyalar:** kütüphane ayar/çizim dosyası yazmaz. `GetSettingsRevision` / `GetDrawingsRevision` değişince host `Export...` ile alıp yazar. Eklenti bunu her karede kontrol eder.
- **Zemin ayarı:** `Open` 'dan hemen sonra (ilk kareden önce) verilirse tam etkili; zemin işçisi ilk karede başlar.
- **CMake (0.22):** `prime_map` SHARED DLL (lzma OBJECT kütüphanesi DLL'in içinde), CRT `/MD` sabit, export makrosu `primemapglobal.h` (`PRIME_MAP_EXPORT`), kutuphane nesnesi `MapLibrary` ; SDK paketi/install yok — tüketim `add_subdirectory` + hedef-adına link (`prime_map_plugin` , `plugin-api-test` , `examples/map_api_test`); `map_tests` / `map_preview` iç sınıfları test ettiği için kaynakla derlemeye devam eder.

13c. Çoklu harita: fabrika, MapInstance, TerrainHub (0.23)

0.22'de birden çok pencere açılabiliyordu ama hepsi aynı katmanlara, çizimlere ve uçağa bakıyordu. 0.23'te "harita başına" olan her şey ayrıldı; ortak kalan sadece veri ve önbellek.

```
prime::MapLibrary (surecte tek)                prime::Map (host'ta unique_ptr)
        |                                              |
   engine/MapData  <------- referans -------  engine/MapInstance  (harita basina)
   - NavDatabase, AirportGeometryStore          - LayerManager + 18 katman nesnesi
   - AirportDetailStore, AirwayDatabase         - Drawings, AircraftData (kaynak: X-Plane / host)
   - TerrainStore (isci) + TerrainHub           - TerrainPalette + palet numarasi
   - prime_map.ini -> sinirlar (ApplyLimits)    - MapWindow (gorunum, mod, UI; XPLM penceresi acik ya da
degil)
   - flight loop: her karede haritalari gezer   - ayar dosyalari: <veri>/maps/<ad>/
```

Neden böyle

- **Katmanlar harita başına nesne:** katmanın görünürlüğü, alt katmanları, zoom aralığı ve çizim önbelleği (GPU mesh'leri) kendi içinde. Paylaşılsaydı bir haritada kapatılan katman diğerinde de kapanırdı. Veri depoları ise `const&` ile paylaşılır: aynı havalimanı iki kez okunmaz.
- **Fabrika (pencere olmadan nesne):** `MapWindow` XPLM penceresi olmadan da bütün durumu tutar; `Create()` pencereyi o durumla açar, `Destroy()` kapatır ama durum kalır. Böylece API'nin her fonksiyonu açılmadan önce de çalışır.
- **Dosya önceliği:** `Open` sırasında önce o anki durum belleğe yazılır (`SaveSettings`), sonra dosya üstüne okunur (`MergeText`), sonra uygulanır. Dosyada olmayan anahtar için varsayılan = o anki değer (kodla verilen).

TerrainHub: zeminin ortak kullanımı

```
TerrainStore (isci) --PopImage----> TerrainHub: dokular (GPU, tek kopya)   <-- TerrainLayer (harita 1, 2
...)
                    --PopVectors--> TerrainHub: vektorler (bellek)         <-- RoadLayer    (harita 1, 2
...)
```

- İşçinin sonuç kuyruğunu "ilk alan" sahiplenseydi iki harita birbirinin dokusunu çalardı. Hub sonuçları tek yerde toplar; her doku onu son çiziminde kullanan haritaların bitleriyle işaretlidir (`BeginDraw` siler, `UseTexture` koyar); bütçe aşılınca sadece işaretsiz dokular, en eskiden başlayarak silinir.
- Doku yükleme hakkı (karede 3) bütün haritalar için ortaktır; flight loop `BeginFrame` ile yeniler. Katmanlar "yeni doku geldi mi"yi `TextureGeneration` sayacından anlar (başka haritanın yüklediği doku da sayılır).
- **Palet numarası** zemin anahtarındadır (`TerrainImageKey::mPalette`): her farklı zemin ayarı `TerrainStore::AddPalette` ile bir kez tanıtılır (`MapData::RegisterPalette` aynı içeriğe aynı numarayı verir). Ayar değişince yeni numarayla yeni dokular istenir: X-Plane'i yeniden başlatmak gerekmez; aynı ayarlı haritalar dokuları paylaşır.
- İstek önceliği isteğin yapıldığı andaki odağa (isteyen haritanın merkezi) göre hesaplanıp istekle saklanır.
- Sınırlar (`MapData::ApplyLimits`): `sınır(n) = taban × (1 + artış% × (n − 1) / 100)` ; disk sınırı (`TerrainStore::SetCacheLimit` , çalışırken değişir, işçi boş anında kırpar), doku bütçesi (`TerrainHub::SetTextureBudget`), işçinin bellek listeleri (`SetMemoryScale`), yüklü havalimanı sayısı.

Diğer

- **Yazı çizici:** `Ui` 'nin yazı çizicisi süreçte tek işaretçi; her pencere çizim / tıklama işinin başında kendisininkini seçer (`MapWindow::UseOwnText`). Yoksa son açılan pencerenin atlası hepsinin yazısını alırdı.
- **Zoom aralığı:** `Layer::SetZoomRange` / `SetSubLayerZoomRange` ; `LayerManager::DrawAll` her çizimde o anki zoom'u katmanlara verir. Katmanlar `Draw` içinde `IsSubLayerDrawn` (kutu + zoom) kullanır; `IsSubLayerVisible` sadece kutudur (panel, kurallar, dosya).
- **Otomatik kayıt:** `MapInstance::Update` revizyon sayaçlarına ve pencere yerine bakar; en sık `autosave_seconds` arayla yazar. Görünüm (merkez / zoom) tek başına yazmayı tetiklemez (uçak takibinde her karede değişir).
- **Yer tutucu:** dokusu hiç gelmemiş kurulu karo Scenery'de ortalama kara tonuyla doldurulur (siyah yerine).
- **Yok etme sırası:** katman yıkıcıları hub'a haber verir; hub katmanlardan sonra yok olmalı (`MapData` haritalardan sonra durur; önizlemede `TerrainHub` `LayerManager` 'dan önce tanımlı).

14. Ayar ve önbellek dosyaları

0.23: dosyaları kütüphane yönetir: `prime_map.ini` (merkezi), `maps/<ad>/map_settings.ini` , `map_drawings.txt` , `terrain_colors.ini` (harita başına), `cache/` (ortak). Anahtarlar: `docs/PrimeMap_API.pdf` bölüm 8–9. Aşağıdaki tablo 0.22 ve öncesinin kökteki dosyalarını anlatır; anahtar adları aynıdır (`[cache]` hariç: o artık `prime_map.ini` 'de).

Hepsi `C:\X-Plane 11\Resources\plugins\prime-map-plugin\` altında (eklenti X-Plane klasöründe başka hiçbir yere yazmaz). INI dosyaları `Settings` sınıfıyla okunur/yazılır.

Dosya	İçerik
`64/win.xpl`	Eklentinin kendisi

`map_settings.ini`	`[window]` yer/boyut/görünür · `[view]` merkez, zoom, takip, kontroller, `mode` · `[tools]` palet · `[layers.scenery]`, `[layers.vfr]`, `[layers.ifr_low]`, `[layers.ifr_high]`, `[layers.dark]` (katman ve `katman.alt` anahtarları); eski `[layers]` Scenery'ye okunur · `[cache] terrain_mb` zemin önbelleği sınırı
`map_drawings.txt`	Kullanıcı çizimleri (metin)
`terrain_colors.ini`	`[colors]`, `[shading]`, `[blend]`, `[warp]`, `[closeup]`, `[relief]`, `[water]`, `[vfr]`, `[palette] version` (bölüm 9.6)
`cache/terrain/`	Zemin önbelleği (silinebilir): `*.terrain` sınıf raster'ı, `*.elevation` tam çözünürlüklü DEM + sea_level, `*.vectors`, `*.nobase`, `xplane_palette.txt`; en çok `terrain_mb` (9.2)
`cache/airports/airport_index.bin`	apt.dat özet indeksi (silinebilir)

15. Dosya dizini

src/ — giriş, pencere, çekirdek

Dosya	Görev
`Plugin.cpp`	XPluginStart/Stop/Enable/Disable; menü "Prime Map Plugin"; komutlar `prime/map/toggle`, `controls`, `diagnostics`, `next_mode`; flight loop; `CreateLayers()`; terrain_colors.ini
`MapWindow.*`	Pencere (pop-out), fare/tekerlek, arayüz olayları, mod değişimi, ayar kaydı, tooltip
`MapView.*`	Web Mercator kamera, lat/lon ↔ boxel, imleç etrafında zoom
`ScreenMath.*`	Ekran koordinatı yardımcıları (dikdörtgen, nokta)
`Layer.h`, `LayerManager.*`	Katman arayüzü, alt katmanlar, DrawContext; liste, aç/kapa, mod başına kayıt
`MapTheme.*`	Harita modları ve renkleri
`LabelPlacer.*`	Etiket/sembol çakışma önleme
`MapFormat.*`	Koordinat/mesafe yazı biçimleri, ızgara ve ölçek aralığı
`Geo.*`, `GeoGrid.*`	Büyük daire hesapları, poligon üçgenleme; 1×1° uzamsal indeks
`Drawings.*`	Kullanıcı çizim modeli ve dosyası
`Settings.*`, `Log.*`, `BackgroundJob.*`, `Color.h`	INI, Log.txt yazımı, tek seferlik arka plan işi, Rgba

src/data/ — veri

Dosya	Görev
`AptLine.*`	apt.dat satırını yerel ayardan bağımsız okuyan alan okuyucu
`AptDatIndexer.*`	Özet indeksi + disk önbelleği
`AptDatParser.*`	Tek havalimanı metni → `AirportDetail`
`AirportGeometry.*`, `AirportGeometryStore.*`	Özet yapısı; özet deposu + ızgara
`AirportSearch.*`	(0.18) Havalimanı arama ve sığdırma zoom'u (Go to airport)
`AirportDetail.h`, `AirportDetailStore.*`	Ayrıntı yapısı; istekle okuyan işçi + LRU
`AirportShapes.*`	Bezier, earcut üçgenleme, çizgi/ışık dizileri
`LocalizerMatch.*`	ILS → pist eşiği
`NavDatabase.*`, `NavLoader.*`	Nav kayıtları + ızgara; SDK'dan dilimli yükleme
`AirwayDatabase.*`	earth_awy.dat
`AircraftData.*`	Uçak konumu/yönü
`SceneryPacks.*`, `SceneryIndex.*`	Sahne sırası; karo → aday DSF'ler
`SevenZip.*`, `DsfReader.*`	7z açma; DSF okuma (arazi adları, üçgenler — yama bayrağı ve düzlem sayısı dahil, DEM)

`TerrainClasses.*`	Arazi türü → sınıf, palet (sürüm 2), rakım rampası (`ElevationColor`), blend / warp / closeup / relief / water / vfr ayarları
`TerrainRaster.*`	Sınıf raster'ı, görüntü üretimi (kaba ve yakın) ve son adım `ComposeImage` , önbellek dosyası
`TerrainElevation.*`	Tam çözünürlüklü DEM, piramit, sea_level, rakım dosyası; 3 × 3 komşulu bikubik örnekleyici
`ElevationCodec.*`	16 bitlik ızgara için satır farkı + LZMA (kayıpsız)
`TerrainRelief.*`	Piksel formülleri: kabartma, VFR bandı, derinlik tonu, kıyı şeridi, kıyı mesafesi ve çizgisi
`WorldTileMap.*`	`dsf_files` : dünyada hangi karonun DSF'i var (deniz / veri yok ayrımı)
`TerrainVectors.*`	DSF orman / yerleşim alanları, yollar, dışlamalar; sadeleştirme, önbellek, alan taraması
`TerrainPaletteSource.*`	Sınıf renklerini X-Plane'in arazi dokularından (DDS) çıkarma
`TerrainTriangles.*`	Yakın zoom için hafif üçgen deposu (parça dizinli)
`TerrainNoise.*` , `LandColorField.*`	Sınır kıvırma gürültüsü; kara renklerinin yumuşak geçişi
`CacheTrim.*`	(0.18) Zemin disk önbelleği sınırı: karo gruplarını en eski kullanılandan siler
`TerrainStore.*`	Zemin işçi thread'i, istek/sonuç kuyrukları, raster / üçgen / rakım LRU, eksik komşu takibi, süre özeti

src/layers/, src/gfx/, src/ui/

Bölüm 7.2 (katmanlar), 11 (gfx) ve 12 (ui) tablolarına bak. Her dosyanın başında Türkçe bir açıklama bloğu vardır (ne yapar, neden, zoom eşikleri, şema); ayrıntı için önce o bloğu oku.

Diğer

Yer	Ne
`tests/tests.cpp`	X-Plane'siz testler (bölüm 16)
`tools/preview/`	`map_preview` : aynı katman kodunu XPLM taklitleriyle resme çizer
`tools/dsfinfo/`	`dsf_info` : bir DSF dosyasının içeriğini döker
`third_party/`	XPSDK (XPLM200..301 açık), LZMA SDK 23.01 (7z açıcı + 0.16'dan beri LZMA sıkıştırıcı, tek thread; `lzma/README.txt`), mapbox earcut 2.2.4 (tek belgelenmiş yama: `README.txt`). Dış kod sadece burada.
`CMakeLists.txt`	Hedefler: `prime-map-plugin` (win.xpl), `map_tests` , `map_preview` , `dsf_info` , `lzma_sdk`

16. Testler ve araçlar

- **`map_tests`** (`-DMAP_TESTS=ON`): çekirdeği X-Plane'siz dener — Mesh geometrisi, MapView gidiş-dönüş ve zoom, LabelPlacer, ayarlar, Drawings, Geo, apt.dat ayrıştırma ve Bezier, AirportShapes, AirwayDatabase satırları, DSF okuma ve overlay sayımı, zemin raster'ı, gürültü ve renk karışımı, temalar, mod başına görünürlük; 0.16: rakım kodlayıcısı, piramit, komşulu örnekleme (karo sınırında dikiş yok), kabartma / VFR bandı / su formülleri, `dsf_files` ve doğrulaması, rakım önbelleği. Hepsi geçmeli.

- **`map_preview`** (`-DMAP_PREVIEW=ON`): eklentinin kendi katman ve arayüz kodunu gerçek X-Plane verisiyle görünmez bir OpenGL penceresinde çizip PPM resim yazar. Örnek: `map_preview --lat 47.45 --lon -122.30 --zoom 12 --mode vfr --out ksea.ppm` (ayrıca `--size --show --hide --panel --describe x,y --demo-drawings --no-ui --terrain-colors ini`). Görsel kontrol için hızlıdır ama gerçek X-Plane denemesinin yerine geçmez.
- **`dsf_info`** : DSF'in özellikleri, dışlama alanları, arazi türleri, üçgen sayıları, yama bayrağı × düzlem sayısı, nesne / poligon / yol ağı envanteri, raster katmanları, aşama süreleri; `--raw` ile açılmış hâlini, `--dump-terrain` ile bir arazi türünün üçgenlerini yazar. DSF biçimiyle ilgili bir şey değiştirmeden önce gerçek dosyaya bununla bak.

17. Hız ve bellek yöntemleri (genel)

Haritanın X-Plane'in FPS'ini düşürmemesi için kullanılan yöntemler; yeni kod yazarken aynı kalıpları izle.

Yöntem	Nerede	Neden
1×1° uzamsal ızgara	`GeoGrid` , `NavDatabase` , `AirportGeometryStore`	Yüz binlerce kayıt var; her karede sadece ekrandaki hücrelere bakılır.
Dilimli yükleme	`NavLoader`	SDK sadece ana thread'den çağrılabildiği için iş flight loop'ta 4 ms'lik dilimlere bölünür; simülatör takılmaz.
İstekle, arka planda hazırlama	`AirportDetailStore` , `TerrainStore`	Katman `Use` / `Request` ile ister, hazır değilse yer tutucu çizer; yavaş iş çizimi bekletmez. En son istenen önce yapılır.
İki seviyeli veri	apt.dat özet + ayrıntı	Yüzlerce MB'lık dosyanın sadece özeti bellekte; ayrıntı ekrana gelen havalimanı için dosyanın o bayt aralığından okunur.
Disk önbellekleri	`airport_index.bin` , `cache/terrain`	Pahalı tarama / DSF açma bir kez yapılır; anahtar yol|boyut|mtime olduğu için kaynak değişince kendiliğinden yenilenir.
LRU sınırları	Ayrıntı (64 havalimanı), zemin dokusu (512 MB), üçgen (2 karo), raster	Bellek dolaşılan alanla sınırsız büyümez; en uzun süredir kullanılmayan atılır.
GPU'da kalıcı şekil	`StaticMesh` + `DrawStatic`	Büyük havalimanı yüzeyleri (yüz binlerce nokta) bir kez yüklenir; her karede sadece ölçek + kaydırma gönderilir.
Harita resmi önbelleği	`MapImageCache`	Dururken harita saniyede ~1 kez, kayarken en çok 15 kez çizilir; aradaki karelerde tek dokulu dörtgen.
Yazı toplu çizimi	`AtlasTextRenderer` , `Canvas::QueueText`	Yüzlerce etiket tek çizim çağrısında; X-Plane'in durum değişikliği etiket başına değil.
Doku yükleme sınırı	`TerrainLayer`	Karede en çok 3 doku yüklenir (takılma olmasın); ince seviye gelene kadar elde olan kaba seviye gösterilir.
Merkezden dışa öncelik	`AirportVisibility`	Ekranın ortasındaki havalimanının ayrıntısı önce istenir.
Zoom eşikleri	Her katmanın .cpp başındaki `k...MinZoom` sabitleri	Uzakta anlamsız ayrıntı (fix, ışık, tabela, boya çizgisi) hiç işlenmez.
Hücreli çakışma testi	`LabelPlacer` (32 boxel hücreler)	Yeni etiket kutusu sadece değdiği hücrelerdeki kutularla karşılaştırılır.

Üçgenleme hızı	`third_party/earcut` (tek yama)	Earcut'ın `isValidDiagonal`'ında ucuz yerel testler pahalı halka taramasından önce yapılır; sonuç aynı, çok delikli apronlarda çok daha hızlı. Ayrıntı `third_party/earcut/README.txt`.
Tek çizim kapısı, toplu çizim	`Canvas`, `Mesh`	Bir katmanın bütün sembolleri tek Mesh'te toplanıp tek çağrıda çizilir.
Rakım piramidi (0.16)	`ElevationGrid`	Uzak seviyeler seyrek nokta okumaz, önceden ortalanmış kaba seviyeyi okur: hem hızlı hem parazitsiz.
Sıkıştırılmış rakım (0.16)	`ElevationCodec`, `*.elevation`	Karo başına 2,9 MB yerine ~0,75 MB; açma ~50 ms. Rakım raster'dan ayrı dosyada: komşu karo için sadece rakım okunur.
Aynı işi iki kez yapmama (0.16)	`TerrainStore::Request`	Üretilmekte olan ya da sonucu kuyrukta bekleyen doku tekrar istenirse yok sayılır.
İki katmanlı rakım belleği (0.16)	`TerrainStore::ElevationFor`	Tam ızgara (~5 MB) en çok 12 karo; uzak zoom için sadece seviye 2+ (~0,5 MB) en çok 64 karo. Tam listeden atılan ızgara silinmez, küçültülüp kaba listeye geçer. İlk sürümde z8.5'te 15 karo × 3×3 komşuluk 12'lik listeyi sürekli boşaltıyor, aynı rakım görünüm başına ~30 kez diskten okunuyordu (ölçüldü).
Hedefli yenileme (0.16)	`mMissingNeighbors`	Sadece komşusu eksik üretilmiş ve o komşusu artık hazır olan doku yeniden üretilir; eski doku yenisi gelene kadar çizilir. Bir kez hazır olup artık bulunamayan komşu beklenmez (sonsuz yenileme olmaz).